

Presentación de Historia de usuario o Historia Técnica

Historia:

466087 [Auditoria errores]: Loguiar en auditoria los mensajes de notificación y fnalización del webhook

Sarlaft

Tabla de contenido

Descripción o contexto de la Historia..... 3

Componentes, módulos o servicios impactados.....4

 Repositorios:.....4

Cambios realizados en la historia.....5

Documentación técnica.....6

Descripción o contexto de la Historia

Se requiere habilitar logs de notificación y finalización del webhook en sherlock mostrando el json enviado al aplicativo destino de la notificación para que se cuente con trazabilidad de los consumos y soporte al usuario

Componentes, módulos o servicios impactados

Para la realización de la Historia se impacto los siguientes micro servicios:

892-sarlaft-function_webhook-mi

Repositorios:

A continuación se presenta los repositorios de los proyectos impactados:

https://dev.azure.com/SuraColombia/Gerencia_Tecnologia/_git/892-sarlaft-function_webhook-mi?version=GBdevelop&a=contents

Cambios realizados en la historia

Dentro de los cambios realizados se mencionan:

892-sarlaft-function_webhook-mi:

Para el este proyecto, se agrega nuevos atributos al Dto el enriquecimiento de datos que tendrá por medio de la cola de Azure bus service y así mismo para la construcción del objeto que se presentará por medio de un servicio REST o QUEUE según sea el caso.

También se generó un Log de pruebas para la verificación de que la información obtenida por medio del bus service de Azure contenga la información nueva.

Documentación técnica

En esta sesión se mostrara pantallas del los cambios realizados:

```
1 usage  Johnathan Monsalve +2
8  @Getter
9  @AllArgsConstructor
10 @NoArgsConstructor
11 @Builder(toBuilder = true)
12 public class MessageControl {
13     private String control;
14     private String mensajeControl;
15     private String estado;
16     private String codigoRespuesta;
17     private String detalleRespuesta;
18 }
19
```

Log de prueba colocado en el flujo de trabajo previo a la salida de información por medio de Queue o por REST.

```
1 usage  ldmunoz@sura.com.co <ldmunoz@sura.com.co> +2
55 @ private Mono<Void> doOperation(Consumidor consumer, MessageIn messageIn) {
56
57     return Mono.just(messageIn).map(messageIn -> Mono<MessageIn>
58         .map(message -> {
59             Gson gson = new Gson();
60             messageLogRepository.logMessage( mensajeInfo: "TEST-WEBHOOK-FUNCTION: ",
61                 nombreMetodo: "notificationFinished", gson.toJson(messageIn));
62
63             switch (consumer.getCdTipoComunicacion()) {
64                 case REST_OPERATION:
65                     return webhookNotificationRestPort.notificationPost(message, consumer);
66                 case QUEUE_OPERATION:
67                     return publisherRabbitMqPort.sendNotificationMessage(message, consumer);
68                 default:
69                     MessageOut messageOut = new MessageOut(messageIn);
70                     messageOut.setMensajeError(ERROR_MESSAGE_NO_TIPO_COMUNICACION);
71                     return messageOut;
72             }
73         }) Mono<MessageOut>
74         .flatMap(mensajeResultado -> sendNotificationCommand.sendCommandDelivered(mensajeResultado)
75         .onErrorResume(Mono::error);
76     }
77
```